DFA Minimizer

Doodlebob

The following is an auto-explainer for DFA Minimization as represented by a series of figures depicting the evolving partitions of the DFAâ€™s state set.

A Deterministic Finite Automaton (DFA) is denoted by the quintuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

and the minimization process works by iteratively refining partitions of the state set until all equivalent states are grouped together. In the minimization process:

- The initial partition divides Q into two blocks: the final states F and the non-final states $Q \setminus F$.
- At each subsequent step, the partition is refined by examining the transitions for every input symbol and grouping together states that behave equivalently.
- This iterative refinement continues until no further splitting is possible, thus yielding a minimal DFA.

1 DFA Definition and Minimization Overview

A Deterministic Finite Automaton is defined as a 5-tuple

$$(Q, \Sigma, \delta, q_0, F)$$

where:

- Q is a finite set of states,
- Σ is a finite input alphabet,
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the start state, and
- $F \subseteq Q$ is the set of final (accepting) states.

The minimization of a DFA entails computing an equivalent DFA with the fewest possible states. The process begins by partitioning the state set into final and non-final states and then repeatedly refining these partitions by identifying states that are indistinguishable with respect to the input symbols. The resulting minimized DFA retains the same language as the original.

2 DFA Transition Function

The DFA's transition function, δ, is defined as follows:

$$\delta(1,\, a) = 2$$
$$\delta(1,\, b) = 4$$

$$\delta(2,\, a) = 4$$
$$\delta(2,\, b) = 3$$

$$\delta(3,\, a) = 3$$
$$\delta(3,\, b) = 3$$

$$\delta(4,\, a) = 2$$
$$\delta(4,\, b) = 5$$

$$\delta(5,\, a) = 6$$
$$\delta(5,\, b) = 5$$

$$\delta(6,\, a) = 4$$
$$\delta(6,\, b) = Z$$

$$\delta(Z,\, a) = Z$$
$$\delta(Z,\, b) = Z$$

3 Partition-Refinement Minimization

The DFA is first minimized by iterative partition refinement. States in the same block are equivalent up to that step's transitions.

Partition Step 1

Explanation:
Initially, the states are split into final and non-final blocks:

- **Final States:** ['3']
- **Non-Final States:** ['1', '2', '4', '5', '6', 'Z']

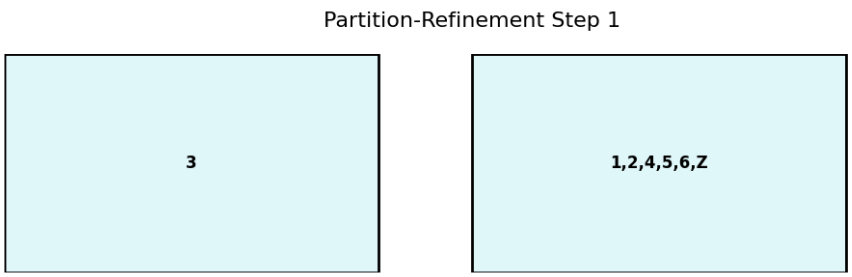

Figure 1: Partition-refinement at step 1

Partition Step 2

Explanation:
This refinement produces Step 2: 3 block(s) — 3; 1,4,5,6,Z; 2.

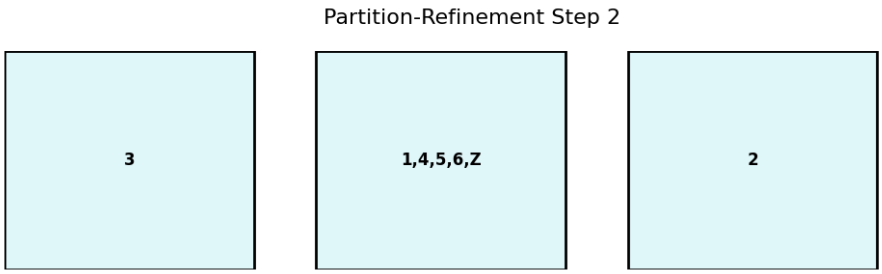

Figure 2: Partition-refinement at step 2

Partition Step 3

Explanation:
This refinement produces Step 3: 4 block(s) — 3; 5,6,Z; 1,4; 2.

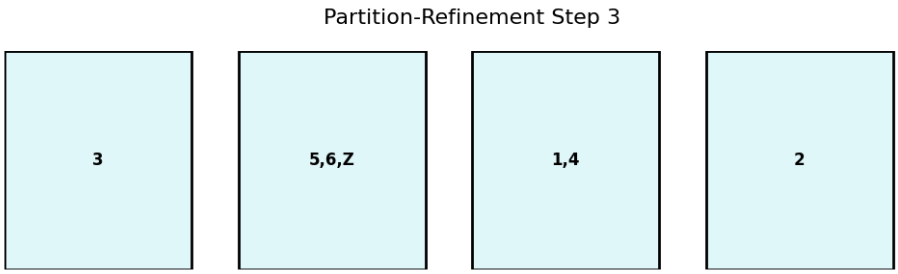

Figure 3: Partition-refinement at step 3

Partition Step 4

Explanation:
This refinement produces Step 4: 6 block(s) — 3; 5,Z; 6; 1; 4; 2.

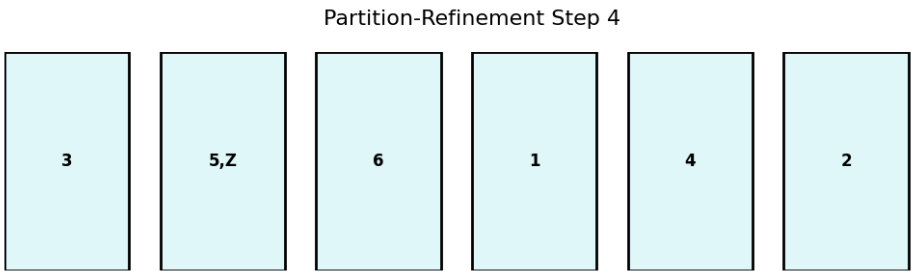

Figure 4: Partition-refinement at step 4

Partition Step 5

Explanation:
This refinement produces Step 5: 7 block(s) — 3; 5; Z; 6; 1; 4; 2.

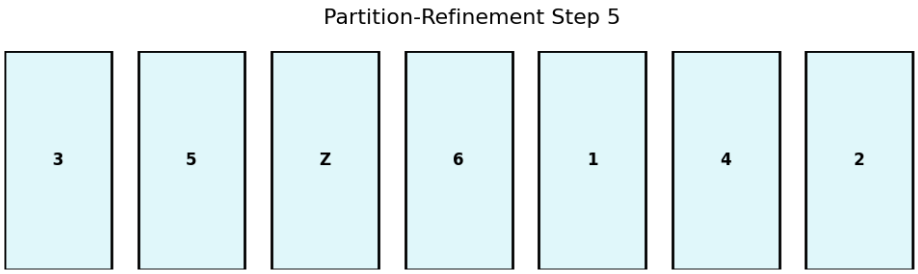

Figure 5: Partition-refinement at step 5

4 Indistinguishability Table

Next, we apply the table-filling (indistinguishability) algorithm. Marked pairs (X) in each triangular table step show which state-pairs have been determined distinguishable.

Table Step 1

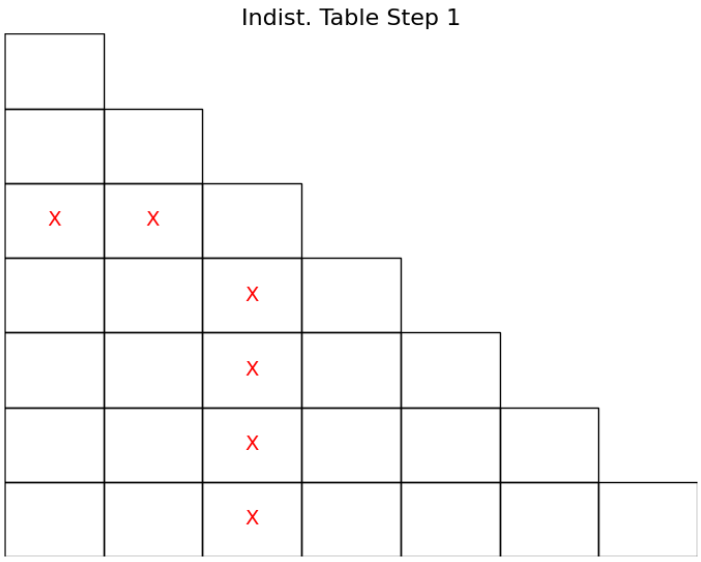

Figure 6: Indistinguishability table at iteration 1

Table Step 2

Table Step 3

Table Step 4

Table Step 5

5 Minimized DFA

Finally, states that remain unmarked and in the same block are merged. The resulting minimized DFA is:

Indist. Table Step 2

X						
X	X					
	X	X				
	X	X				
	X	X				
	X	X				

Figure 7: Indistinguishability table at iteration 2

Indist. Table Step 3

X						
X	X					
	X	X				
X	X	X	X			
X	X	X	X			
X	X	X	X			

Figure 8: Indistinguishability table at iteration 3

Indist. Table Step 4

X						
X	X					
X	X	X				
X	X	X	X			
X	X	X	X	X		
X	X	X	X		X	

Figure 9: Indistinguishability table at iteration 4

Indist. Table Step 5

X						
X	X					
X	X	X				
X	X	X	X			
X	X	X	X	X		
X	X	X	X	X	X	

Figure 10: Indistinguishability table at iteration 5

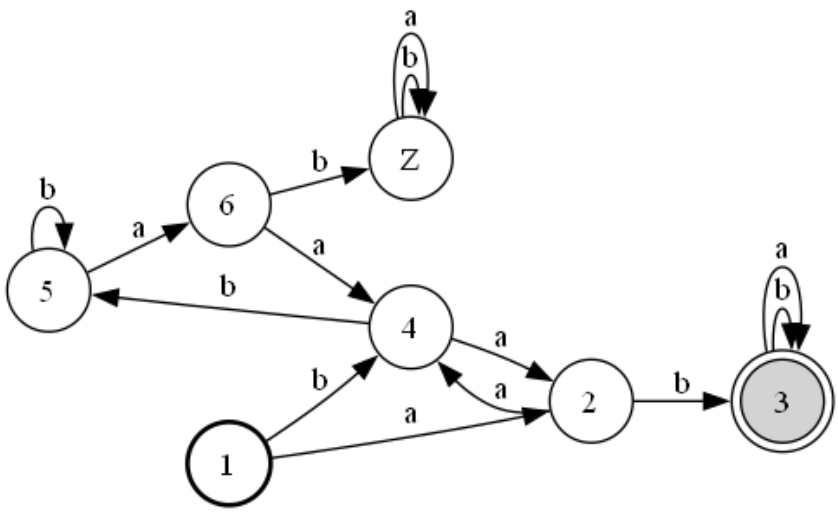

Figure 11: Diagram of the minimized DFA